

Punto 4.12 - Estrategias de optimización y checklist de rendimiento

Práctica

Ejercicio: Aplicar el checklist de rendimiento al proyecto AceriaData. Crear un método que recorra todas las consultas del repositorio y verifique las condiciones del checklist. Medir el impacto de las optimizaciones. Documentar las decisiones. Cerrar el Módulo 4 con un caso práctico completo de optimización.

Contexto del proyecto: En el punto 4.11 se estudió el diagnóstico, configurando logs detallados y observadores de diagnóstico. En este punto se consolidan todas las técnicas del Módulo 4 en un checklist de rendimiento que se aplica al proyecto AceriaData. Esta consolidación cierra el Módulo 4 y prepara el proyecto para el Módulo 5.

Paso 1: Abrir el proyecto

```
cd AceriaData
cd src/AceriaData.Console
```

cd AceriaData → entra en la carpeta raíz del proyecto. cd src/AceriaData.Console → entra en la carpeta del proyecto de consola.

Paso 2: Añadir los métodos del checklist a la interfaz del repositorio

Modificar src/AceriaData.Application/Interfaces/IOrdenRepositorio.cs:

```
using AceriaData.Application.Dtos;
using AceriaData.Domain.Entities;

namespace AceriaData.Application.Interfaces;

public interface IOrdenRepositorio
{
    // ... métodos existentes ...

    List<OrdenFabricacion> ObtenerSinOptimizar();
    List<OrdenFabricacion> ObtenerOptimizado();
    List<OrdenResumenDto> ObtenerResumenOptimizado();
    List<OrdenFabricacion> ObtenerConRelacionesOptimizado();

    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
}
```

Línea 1: using AceriaData.Application.Dtos; → importa los DTOs.

Línea 2: using AceriaData.Domain.Entities; → importa las entidades.

Línea 4: namespace AceriaData.Application.Interfaces; → declara el espacio de nombres.

Línea 6: public interface IOrdenRepositorio → declara la interfaz.

Línea 8: // ... métodos existentes ... → comentario que indica que se mantienen los métodos anteriores.

Línea 10: List ObtenerSinOptimizar(); → método sin optimización.

Línea 11: List ObtenerOptimizado(); → método con optimización de tracking y proyección.

Línea 12: List ObtenerResumenOptimizado(); → método con proyección a DTO.

Línea 13: List ObtenerConRelacionesOptimizado(); → método con Include y AsSplitQuery.

Línea 15: void Agregar(OrdenFabricacion orden); → método que agrega una orden.

Línea 16: void Eliminar(OrdenFabricacion orden); → método que elimina una orden.

Error común: si los métodos devuelven IQueryable, la capa de aplicación puede componer consultas que el repositorio no controla. Se deben devolver listas o valores concretos.

| Paso 3: Implementar los métodos del checklist en el repositorio

Modificar src/AceriaData.Infrastructure/Repositories/OrdenRepositorio.cs:

```

public List<OrdenFabricacion> ObtenerSinOptimizar()
{
    return _context.OrdenesFabricacion
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}

/// <summary>
/// Obtiene las órdenes con optimización de tracking.
/// Usa AsNoTracking porque es una consulta de solo lectura.
/// </summary>
public List<OrdenFabricacion> ObtenerOptimizado()
{
    return _context.OrdenesFabricacion
        .AsNoTracking()
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}

/// <summary>
/// Obtiene los resúmenes de las órdenes.
/// Usa proyección para reducir el volumen de datos transferidos.
/// Usa AsNoTracking porque es una consulta de solo lectura.
/// </summary>
public List<OrdenResumenDto> ObtenerResumenOptimizado()
{
    return _context.OrdenesFabricacion
        .AsNoTracking()
        .OrderBy(o => o.NumeroOrden)
        .Select(o => new OrdenResumenDto
        {
            NumeroOrden = o.NumeroOrden,
            Cliente = o.Cliente,
            Estado = o.Estado,
            FechaCreacion = o.FechaCreacion
        })
        .ToList();
}

/// <summary>
/// Obtiene las órdenes con planchas y detalle.
/// Usa Include para evitar N+1.
/// Usa AsSplitQuery para evitar el producto cartesiano.
/// Usa AsNoTrackingWithIdentityResolution para evitar instancias duplicadas.
/// </summary>
public List<OrdenFabricacion> ObtenerConRelacionesOptimizado()
{
    return _context.OrdenesFabricacion
        .AsNoTrackingWithIdentityResolution()
        .Include(o => o.Planchas)
        .Include(o => o.Detalle)
        .AsSplitQuery()
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}
  
```

Línea 1: public List ObtenerSinOptimizar() → declara el método sin optimización.

Línea 3: return _context.OrdenesFabricacion → inicia la consulta.

Línea 4: `.OrderBy(o => o.NumeroOrden)` → ordena por número de orden.

Línea 5: `.ToList();` → materializa la consulta con tracking.

Línea 8: `///`

→ inicio del comentario XML.

Línea 9: `///` Obtiene las órdenes con optimización de tracking. → descripción del método.

Línea 10: `///` Usa `AsNoTracking` porque es una consulta de solo lectura. → explicación del uso de `AsNoTracking`.

Línea 11: `///`

→ cierre del comentario XML.

Línea 12: `public List ObtenerOptimizado()` → declara el método optimizado.

Línea 14: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 15: `.AsNoTracking()` → aplica `AsNoTracking`.

Línea 16: `.OrderBy(o => o.NumeroOrden)` → ordena por número de orden.

Línea 17: `.ToList();` → materializa la consulta sin tracking.

Línea 20: `///`

→ inicio del comentario XML.

Línea 21: `///` Obtiene los resúmenes de las órdenes. → descripción del método.

Línea 22: `///` Usa proyección para reducir el volumen de datos transferidos. → explicación del uso de proyección.

Línea 23: `///` Usa `AsNoTracking` porque es una consulta de solo lectura. → explicación del uso de `AsNoTracking`.

Línea 24: `///`

→ cierre del comentario XML.

Línea 25: `public List ObtenerResumenOptimizado()` → declara el método.

Línea 27: `return _context.OrdenesFabricacion` → inicia la consulta.

Línea 28: `.AsNoTracking()` → aplica `AsNoTracking`.

Línea 29: `.OrderBy(o => o.NumeroOrden)` → ordena por número de orden.

Línea 30: `.Select(o => new OrdenResumenDto` → proyecta al DTO.

Línea 31: `NumeroOrden = o.NumeroOrden,` → asigna el número.

Línea 32: `Cliente = o.Cliente,` → asigna el cliente.

Línea 33: `Estado = o.Estado,` → asigna el estado.

Línea 34: `FechaCreacion = o.FechaCreacion` → asigna la fecha.

Línea 35: `}` → cierra la proyección.

Línea 36: `.ToList();` → materializa la consulta.

Línea 39: `///`

→ inicio del comentario XML.

Línea 40: `///` Obtiene las órdenes con planchas y detalle. → descripción del método.

Línea 41: `///` Usa `Include` para evitar N+1. → explicación del uso de `Include`.

Línea 42: /// Usa AsSplitQuery para evitar el producto cartesiano. → explicación del uso de AsSplitQuery.

Línea 43: /// Usa AsNoTrackingWithIdentityResolution para evitar instancias duplicadas. → explicación del uso de AsNoTrackingWithIdentityResolution.

Línea 44: ///

→ cierre del comentario XML.

Línea 45: public List ObtenerConRelacionesOptimizado() → declara el método.

Línea 47: return _context.OrdenesFabricacion → inicia la consulta.

Línea 48: .AsNoTrackingWithIdentityResolution() → aplica resolución de identidad.

Línea 49: .Include(o => o.Planchas) → incluye la colección de planchas.

Línea 50: .Include(o => o.Detalle) → incluye la referencia al detalle.

Línea 51: .AsSplitQuery() → divide la consulta en varias.

Línea 52: .OrderBy(o => o.NumeroOrden) → ordena por número de orden.

Línea 53: .ToList(); → materializa la consulta.

Error común: si se olvida el comentario XML, otros desarrolladores no entienden por qué se aplicó la técnica. Se deben documentar todas las decisiones.

| Paso 4: Crear el caso de uso del checklist de rendimiento

Crear el archivo src/AceriaData.Application/UseCases/ChecklistRendimientoUseCase.cs:

```
using System.Diagnostics;
using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases;

public class ChecklistRendimientoUseCase
{
    private readonly IUnidadDeTrabajo _unidad;

    public ChecklistRendimientoUseCase(IUnidadDeTrabajo unidad)
    {
        _unidad = unidad;
    }

    public void Ejecutar()
    {
        Console.WriteLine("=== CHECKLIST DE RENDIMIENTO ===");

        MostrarChecklist();
        CompararSinOptimizarVsOptimizado();
        CompararEntidadesVsProyeccion();
        CompararSinIncludeVsConInclude();
    }

    private void MostrarChecklist()
    {
        Console.WriteLine("\n-- Checklist de rendimiento --");
        Console.WriteLine("1. ¿Se usa AsNoTracking en consultas de solo lectura?");
        Console.WriteLine("2. ¿Se proyectan solo las columnas necesarias?");
        Console.WriteLine("3. ¿Se evita el problema N+1 con Include?");
        Console.WriteLine("4. ¿Se evita el producto cartesiano con AsSplitQuery?");
        Console.WriteLine("5. ¿Se aplican filtros y paginación en el servidor?");
        Console.WriteLine("6. ¿Se evitan funciones en Where que impidan índices?");
        Console.WriteLine("7. ¿Se usan Compiled Queries en consultas frecuentes?");
        Console.WriteLine("8. ¿Se miden los tiempos y se cuentan las consultas?");
    }

    private void CompararSinOptimizarVsOptimizado()
    {
    }
}
```

```

        Console.WriteLine("\n--- Sin optimizar vs optimizado (tracking) ---");

        var cronometroSin = Stopwatch.StartNew();
        var ordenesSin = _unidad.Ordenes.ObtenerSinOptimizar();
        cronometroSin.Stop();

        var cronometroCon = Stopwatch.StartNew();
        var ordenesCon = _unidad.Ordenes.ObtenerOptimizado();
        cronometroCon.Stop();

        Console.WriteLine($"Sin optimizar: {cronometroSin.ElapsedMilliseconds} ms | Órdenes: {ordenesSin.Count}");
        Console.WriteLine($"Optimizado: {cronometroCon.ElapsedMilliseconds} ms | Órdenes: {ordenesCon.Count}");
    }

    private void CompararEntidadesVsProyeccion()
    {
        Console.WriteLine("\n--- Entidades completas vs proyección ---");

        var cronometroEntidades = Stopwatch.StartNew();
        var ordenesEntidades = _unidad.Ordenes.ObtenerOptimizado();
        cronometroEntidades.Stop();

        var cronometroProyeccion = Stopwatch.StartNew();
        var resúmenes = _unidad.Ordenes.ObtenerResumenOptimizado();
        cronometroProyeccion.Stop();

        Console.WriteLine($"Entidades completas: {cronometroEntidades.ElapsedMilliseconds} ms | Órdenes: {ordenesEntidades.Count}");
        Console.WriteLine($"Proyección: {cronometroProyeccion.ElapsedMilliseconds} ms | Resúmenes: {resúmenes.Count}");
    }

    private void CompararSinIncludeVsConInclude()
    {
        Console.WriteLine("\n--- Sin Include vs con Include y AsSplitQuery ---");

        var cronometroSinInclude = Stopwatch.StartNew();
        var ordenesSinInclude = _unidad.Ordenes.ObtenerOptimizado();
        cronometroSinInclude.Stop();

        var cronometroConInclude = Stopwatch.StartNew();
        var ordenesConInclude = _unidad.Ordenes.ObtenerConRelacionesOptimizado();
        cronometroConInclude.Stop();

        Console.WriteLine($"Sin Include: {cronometroSinInclude.ElapsedMilliseconds} ms | Órdenes: {ordenesSinInclude.Count}");
        Console.WriteLine($"Con Include y AsSplitQuery: {cronometroConInclude.ElapsedMilliseconds} ms | Órdenes: {ordenesConInclude.Count}");
    }
}

```

Línea 1: using System.Diagnostics; → importa Stopwatch.

Línea 2: using AceriaData.Application.Interfaces; → importa las interfaces de la aplicación.

Línea 4: namespace AceriaData.Application.UseCases; → declara el espacio de nombres.

Línea 6: public class ChecklistRendimientoUseCase → declara el caso de uso.

Línea 8: private readonly IUnidadDeTrabajo _unidad; → campo de la unidad de trabajo.

Línea 10: public ChecklistRendimientoUseCase(IUnidadDeTrabajo unidad) → constructor.

Línea 12: _unidad = unidad; → asigna el parámetro al campo.

Línea 15: public void Ejecutar() → declara el método principal.

Línea 17: Console.WriteLine("=== CHECKLIST DE RENDIMIENTO ==="); → muestra la cabecera.

Línea 19: `MostrarChecklist();` → llama al método que muestra el checklist.

Línea 20: `CompararSinOptimizarVsOptimizado();` → llama al método de comparación de tracking.

Línea 21: `CompararEntidadesVsProyeccion();` → llama al método de comparación de proyección.

Línea 22: `CompararSinIncludeVsConInclude();` → llama al método de comparación de Include.

Línea 25: `private void MostrarChecklist()` → declara el método.

Línea 27: `Console.WriteLine("\n--- Checklist de rendimiento ---");` → muestra la cabecera.

Línea 28: `Console.WriteLine("1. ¿Se usa AsNoTracking en consultas de solo lectura?");` → muestra la primera comprobación.

Línea 29: `Console.WriteLine("2. ¿Se proyectan solo las columnas necesarias?");` → muestra la segunda comprobación.

Línea 30: `Console.WriteLine("3. ¿Se evita el problema N+1 con Include?");` → muestra la tercera comprobación.

Línea 31: `Console.WriteLine("4. ¿Se evita el producto cartesiano con AsSplitQuery?");` → muestra la cuarta comprobación.

Línea 32: `Console.WriteLine("5. ¿Se aplican filtros y paginación en el servidor?");` → muestra la quinta comprobación.

Línea 33: `Console.WriteLine("6. ¿Se evitan funciones en Where que impidan índices?");` → muestra la sexta comprobación.

Línea 34: `Console.WriteLine("7. ¿Se usan Compiled Queries en consultas frecuentes?");` → muestra la séptima comprobación.

Línea 35: `Console.WriteLine("8. ¿Se miden los tiempos y se cuentan las consultas?");` → muestra la octava comprobación.

Línea 38: `private void CompararSinOptimizarVsOptimizado()` → declara el método.

Línea 40: `Console.WriteLine("\n--- Sin optimizar vs optimizado (tracking) ---");` → muestra la cabecera.

Línea 42: `var cronometroSin = Stopwatch.StartNew();` → inicia el cronómetro.

Línea 43: `var ordenesSin = _unidad Ordenes.ObtenerSinOptimizar();` → carga las órdenes sin optimizar.

Línea 44: `cronometroSin.Stop();` → detiene el cronómetro.

Línea 46: `var cronometroCon = Stopwatch.StartNew();` → inicia el cronómetro.

Línea 47: `var ordenesCon = _unidad Ordenes.ObtenerOptimizado();` → carga las órdenes optimizadas.

Línea 48: `cronometroCon.Stop();` → detiene el cronómetro.

Línea 50: `Console.WriteLine($"Sin optimizar: {cronometroSin.ElapsedMilliseconds} ms | Órdenes: {ordenesSin.Count}");` → muestra el tiempo sin optimizar.

Línea 51: `Console.WriteLine($"Optimizado: {cronometroCon.ElapsedMilliseconds} ms | Órdenes: {ordenesCon.Count}");` → muestra el tiempo optimizado.

Línea 54: `private void CompararEntidadesVsProyeccion()` → declara el método.

Línea 56: `Console.WriteLine("\n--- Entidades completas vs proyección ---");` → muestra la cabecera.

Línea 58: `var cronometroEntidades = Stopwatch.StartNew();` → inicia el cronómetro.

Línea 59: `var ordenesEntidades = _unidad Ordenes.ObtenerOptimizado();` → carga las entidades completas.

Línea 60: `cronometroEntidades.Stop();` → detiene el cronómetro.

Línea 62: `var cronometroProyeccion = Stopwatch.StartNew();` → inicia el cronómetro.

Línea 63: `var resúmenes = _unidad Ordenes.ObtenerResumenOptimizado();` → carga los resúmenes.

Línea 64: `cronometroProyeccion.Stop();` → detiene el cronómetro.

Línea 66: `Console.WriteLine($"Entidades completas: {cronometroEntidades.ElapsedMilliseconds} ms | Órdenes: {ordenesEntidades.Count}");` → muestra el tiempo de entidades completas.

Línea 67: `Console.WriteLine($"Proyección: {cronometroProyeccion.ElapsedMilliseconds} ms | Resúmenes: {resúmenes.Count}");` → muestra el tiempo de proyección.

Línea 70: `private void CompararSinIncludeVsConInclude()` → declara el método.

Línea 72: `Console.WriteLine("\n--- Sin Include vs con Include y AsSplitQuery ---");` → muestra la cabecera.

Línea 74: `var cronometroSinInclude = Stopwatch.StartNew();` → inicia el cronómetro.

Línea 75: `var ordenesSinInclude = _unidad Ordenes.ObtenerOptimizado();` → carga las órdenes sin Include.

Línea 76: `cronometroSinInclude.Stop();` → detiene el cronómetro.

Línea 78: `var cronometroConInclude = Stopwatch.StartNew();` → inicia el cronómetro.

Línea 79: `var ordenesConInclude = _unidad Ordenes.ObtenerConRelacionesOptimizado();` → carga las órdenes con Include.

Línea 80: `cronometroConInclude.Stop();` → detiene el cronómetro.

Línea 82: `Console.WriteLine($"Sin Include: {cronometroSinInclude.ElapsedMilliseconds} ms | Órdenes: {ordenesSinInclude.Count}");` → muestra el tiempo sin Include.

Línea 83: `Console.WriteLine($"Con Include y AsSplitQuery: {cronometroConInclude.ElapsedMilliseconds} ms | Órdenes: {ordenesConInclude.Count}");` → muestra el tiempo con Include.

Error común: si se ejecutan las consultas en el mismo `DbContext`, las entidades de las consultas anteriores permanecen en el `Change Tracker`. Se debe usar un `DbContext` distinto para cada consulta si se quiere medir el tiempo de forma aislada.

| Paso 5: Registrar el caso de uso en el contenedor

Modificar `src/AceriaData.Console/Program.cs`:

```
services.AddScoped<ChecklistRendimientoUseCase>();
```

Línea 1: `services.AddScoped();` → registra el caso de uso con ciclo de vida `Scoped`.

Error común: si se registra con `Singleton`, se comparte la misma instancia del `DbContext` entre todos los ámbitos y se producen problemas de concurrencia.

| Paso 6: Llamar al caso de uso desde la consola

Modificar el método `Main`:

```
using (var scope = _provider.CreateScope())
{
    var useCase = scope.ServiceProvider.GetRequiredService<ChecklistRendimientoUseCase>();
    useCase.Ejecutar();
}
```

Línea 1: `using (var scope = _provider.CreateScope())` → crea un ámbito.

Línea 3: `var useCase = scope.ServiceProvider.GetRequiredService();` → resuelve el caso de uso.

Línea 4: `useCase.Ejecutar();` → ejecuta el caso de uso.

Error común: si se resuelve el caso de uso desde el proveedor raíz en lugar de desde un ámbito, se lanza una excepción `InvalidOperationException`.

Paso 7: Insertar datos de prueba con varias órdenes y planchas

Asegurarse de que hay datos en la base de datos. Modificar el método `Main` para insertar órdenes y planchas:

```
using (var scope = _provider.CreateScope())
{
    var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
    context.Database.EnsureDeleted();
    context.Database.Migrate();

    var ordenes = new List<OrdenFabricacion>();
    for (int i = 1; i <= 20; i++)
    {
        ordenes.Add(new OrdenFabricacion
        {
            NumeroOrden = $"OF-2024-{i:D4}",
            Cliente = i % 2 == 0 ? "Constructora del Norte" : "Constructora del Sur",
            Estado = i % 3 == 0 ? "EnProceso" : "Pendiente",
            FechaCreacion = new DateTime(2024, 1, 1).AddDays(i)
        });
    }

    context.OrdenesFabricacion.AddRange(ordenes);
    context.SaveChanges();

    var planchas = new List<PlanchaAcero>();
    foreach (var orden in ordenes)
    {
        planchas.Add(new PlanchaAcero { OrdenId = orden.Id, Espesor = 10.5, Ancho = 1500, Largo = 3000,
        Peso = 370.5m, Activa = true });
    }

    context.PlanchasAcero.AddRange(planchas);
    context.SaveChanges();
}
```

Línea 1: `using (var scope = _provider.CreateScope())` → crea un ámbito.

Línea 3: `var context = scope.ServiceProvider.GetRequiredService();` → resuelve el `DbContext`.

Línea 4: `context.Database.EnsureDeleted();` → elimina la base de datos.

Línea 5: `context.Database.Migrate();` → aplica las migraciones.

Línea 7: `var ordenes = new List();` → crea la lista de órdenes.

Línea 8: `for (int i = 1; i <= 20; i++)` → repite veinte veces.

Línea 10: `ordenes.Add(new OrdenFabricacion` → crea una orden.

Línea 12: `NumeroOrden = $"OF-2024-{i:D4}"`, → asigna el número.

Línea 13: `Cliente = i % 2 == 0 ? "Constructora del Norte" : "Constructora del Sur"`, → asigna el cliente.

Línea 14: `Estado = i % 3 == 0 ? "EnProceso" : "Pendiente"`, → asigna el estado.

Línea 15: `FechaCreacion = new DateTime(2024, 1, 1).AddDays(i)` → asigna la fecha.

Línea 16: `});` → cierra la orden.

Línea 17: `}` → cierra el bucle.

Línea 19: `context.OrdenesFabricacion.AddRange(ordenes);` → registra las órdenes.

Línea 20: `context.SaveChanges();` → inserta las órdenes.

Línea 22: `var planchas = new List();` → crea la lista de planchas.

Línea 23: `foreach (var orden in ordenes)` → itera sobre las órdenes.

Línea 25: `planchas.Add(new PlanchaAcero { ... });` → crea una plancha.

Línea 26: `}` → cierra el bucle.

Línea 28: `context.PlanchasAcero.AddRange(planchas);` → registra las planchas.

Línea 29: `context.SaveChanges();` → inserta las planchas.

Error común: si se insertan las planchas antes que las órdenes, la clave foránea `OrdenId` no tiene un valor válido y se produce una violación de integridad referencial.

Paso 8: Ejecutar el proyecto

```
dotnet run
```

`dotnet run` → compila y ejecuta el proyecto.

Resultado esperado: aparecen las secciones con el checklist y las comparaciones de rendimiento. Se observa el tiempo de cada técnica.

Paso 9: Analizar la salida

La salida del programa muestra información como la siguiente:

```
=== CHECKLIST DE RENDIMIENTO ===

--- Checklist de rendimiento ---
1\ ¿Se usa AsNoTracking en consultas de solo lectura?
2\ ¿Se proyectan solo las columnas necesarias?
3\ ¿Se evita el problema N+1 con Include?
4\ ¿Se evita el producto cartesiano con AsSplitQuery?
5\ ¿Se aplican filtros y paginación en el servidor?
6\ ¿Se evitan funciones en Where que impidan índices?
7\ ¿Se usan Compiled Queries en consultas frecuentes?
8\ ¿Se miden los tiempos y se cuentan las consultas?

--- Sin optimizar vs optimizado (tracking) ---
Sin optimizar: 55 ms | Órdenes: 20
Optimizado: 45 ms | Órdenes: 20

--- Entidades completas vs proyección ---
Entidades completas: 45 ms | Órdenes: 20
Proyección: 38 ms | Resúmenes: 20

--- Sin Include vs con Include y AsSplitQuery ---
Sin Include: 45 ms | Órdenes: 20
Con Include y AsSplitQuery: 58 ms | Órdenes: 20
```

La primera sección muestra el checklist. La segunda sección muestra la comparación sin optimizar vs optimizado. La tercera sección muestra la comparación de entidades completas vs proyección. La cuarta sección muestra la comparación sin Include vs con Include.

Observaciones: la optimización de tracking es más rápida. La proyección es más rápida que las entidades completas. La consulta con Include y AsSplitQuery es más lenta porque carga más datos, pero evita el problema N+1 en escenarios reales. El beneficio del Include se ve cuando se necesitan las entidades relacionadas.

Paso 10: Diagnosticar un error común

Modificar el método `ObtenerConRelacionesOptimizado` para eliminar `AsSplitQuery`:

```
public List<OrdenFabricacion> ObtenerConRelacionesOptimizado()
{
    return _context.OrdenesFabricacion
        .AsNoTrackingWithIdentityResolution()
        .Include(o => o.Planchas)
        .Include(o => o.Detalle)
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}
```

Resultado esperado: el código funciona, pero la consulta genera un producto cartesiano porque se incluyen dos colecciones. El número de filas transferidas es mayor que el número de planchas más el número de detalles.

Solución: añadir AsSplitQuery para dividir la consulta en varias.

```
public List<OrdenFabricacion> ObtenerConRelacionesOptimizado()
{
    return _context.OrdenesFabricacion
        .AsNoTrackingWithIdentityResolution()
        .Include(o => o.Planchas)
        .Include(o => o.Detalle)
        .AsSplitQuery()
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}
```

Resultado esperado con la solución: la consulta se divide en varias y no se produce el producto cartesiano.

Errores comunes del ejercicio

Error	Causa	Solución
Tracking innecesario	No se aplica AsNoTracking	Aplicar AsNoTracking en consultas de solo lectura
Over-fetching	Se cargan entidades completas	Usar proyecciones
N+1	No se aplica Include	Aplicar Include en consultas con relaciones
Producto cartesiano	Se incluyen varias colecciones sin AsSplitQuery	Aplicar AsSplitQuery
Consultas en memoria	Se materializa antes de filtrar	Filtrar antes de materializar
Funciones en Where	Se aplican funciones sobre columnas	Comparar directamente o usar collation
Falta de paginación	Se cargan todos los resultados	Aplicar Skip/Take o keyset pagination
Compilación repetida	No se usan Compiled Queries	Aplicar Compiled Queries en consultas frecuentes
Sin diagnóstico	No se miden los tiempos	Configurar logging y observadores

Reto resuelto: Aplicar el checklist a una consulta completa

Reto: Tomar una consulta sin optimizar que cargue las órdenes con sus planchas y detalles, y aplicar el checklist completo: AsNoTracking, proyección, Include, AsSplitQuery, filtro, paginación y documentación. Medir el tiempo antes y después.

Solución paso a paso:

Paso 1: Consulta sin optimizar:

```
public List<OrdenFabricacion> ObtenerSinOptimizarCompleta()
{
    return _context.OrdenesFabricacion
        .Include(o => o.Planchas)
        .Include(o => o.Detalle)
        .ToList();
}
```

Línea 1: public List ObtenerSinOptimizarCompleta() → declara el método.

Línea 3: return _context.OrdenesFabricacion → inicia la consulta.

Línea 4: .Include(o => o.Planchas) → incluye la colección de planchas.

Línea 5: .Include(o => o.Detalle) → incluye la referencia al detalle.

Línea 6: .ToList(); → materializa la consulta.

Paso 2: Consulta optimizada con el checklist completo:

```
/// <summary>
/// Obtiene las órdenes pendientes con planchas y detalle.
/// Usa AsNoTrackingWithIdentityResolution para evitar tracking y duplicados.
/// Usa Include para evitar N+1.
/// Usa AsSplitQuery para evitar el producto cartesiano.
/// Usa filtro por estado para reducir el número de filas.
/// Usa paginación para limitar el número de resultados.
/// </summary>
public List<OrdenFabricacion> ObtenerOptimizadoCompleta(int pagina, int tamanoPagina)
{
    return _context.OrdenesFabricacion
        .AsNoTrackingWithIdentityResolution()
        .Where(o => o.Estado == "Pendiente")
        .OrderBy(o => o.FechaCreacion)
        .ThenBy(o => o.Id)
        .Skip((pagina - 1) * tamanoPagina)
        .Take(tamanoPagina)
        .Include(o => o.Planchas)
        .Include(o => o.Detalle)
        .AsSplitQuery()
        .ToList();
}
```

Línea 1: ///

→ inicio del comentario XML.

Línea 2: /// Obtiene las órdenes pendientes con planchas y detalle. → descripción del método.

Línea 3: /// Usa AsNoTrackingWithIdentityResolution para evitar tracking y duplicados. → explicación del uso de AsNoTrackingWithIdentityResolution.

Línea 4: /// Usa Include para evitar N+1. → explicación del uso de Include.

Línea 5: /// Usa AsSplitQuery para evitar el producto cartesiano. → explicación del uso de AsSplitQuery.

Línea 6: /// Usa filtro por estado para reducir el número de filas. → explicación del filtro.

Línea 7: /// Usa paginación para limitar el número de resultados. → explicación de la paginación.

Línea 8: ///

→ cierre del comentario XML.

Línea 9: public List ObtenerOptimizadoCompleta(int pagina, int tamanoPagina) → declara el método.

Línea 11: return _context.OrdenesFabricacion → inicia la consulta.

Línea 12: `.AsNoTrackingWithIdentityResolution()` → aplica resolución de identidad.

Línea 13: `.Where(o => o.Estado == "Pendiente")` → filtra por estado.

Línea 14: `.OrderBy(o => o.FechaCreacion)` → ordena por fecha.

Línea 15: `.ThenBy(o => o.Id)` → ordena por Id.

Línea 16: `.Skip((pagina - 1) * tamanoPagina)` → salta las páginas anteriores.

Línea 17: `.Take(tamanoPagina)` → toma el tamaño de la página.

Línea 18: `.Include(o => o.Planchas)` → incluye la colección de planchas.

Línea 19: `.Include(o => o.Detalle)` → incluye la referencia al detalle.

Línea 20: `.AsSplitQuery()` → divide la consulta en varias.

Línea 21: `.ToList();` → materializa la consulta.

| Paso 3: Medir el tiempo antes y después:

```
private void CompararCompleta()
{
    Console.WriteLine("\n--- Completa sin optimizar vs optimizada ---");

    var cronometroSin = Stopwatch.StartNew();
    var ordenesSin = _unidad.Ordenes.ObtenerSinOptimizarCompleta();
    cronometroSin.Stop();

    var cronometroCon = Stopwatch.StartNew();
    var ordenesCon = _unidad.Ordenes.ObtenerOptimizadoCompleta(1, 10);
    cronometroCon.Stop();

    Console.WriteLine($"Sin optimizar: {cronometroSin.ElapsedMilliseconds} ms | Órdenes: {ordenesSin.Count}");
    Console.WriteLine($"Optimizado: {cronometroCon.ElapsedMilliseconds} ms | Órdenes: {ordenesCon.Count}");
}
```

Línea 1: `private void CompararCompleta()` → declara el método.

Línea 3: `Console.WriteLine("\n--- Completa sin optimizar vs optimizada ---");` → muestra la cabecera.

Línea 5: `var cronometroSin = Stopwatch.StartNew();` → inicia el cronómetro.

Línea 6: `var ordenesSin = _unidad.Ordenes.ObtenerSinOptimizarCompleta();` → carga sin optimizar.

Línea 7: `cronometroSin.Stop();` → detiene el cronómetro.

Línea 9: `var cronometroCon = Stopwatch.StartNew();` → inicia el cronómetro.

Línea 10: `var ordenesCon = _unidad.Ordenes.ObtenerOptimizadoCompleta(1, 10);` → carga optimizada.

Línea 11: `cronometroCon.Stop();` → detiene el cronómetro.

Línea 13: `Console.WriteLine($"Sin optimizar: {cronometroSin.ElapsedMilliseconds} ms | Órdenes: {ordenesSin.Count}");` → muestra el tiempo sin optimizar.

Línea 14: `Console.WriteLine($"Optimizado: {cronometroCon.ElapsedMilliseconds} ms | Órdenes: {ordenesCon.Count}");` → muestra el tiempo optimizado.

| Paso 4: Llamar al método desde Ejecutar:

```
CompararCompleta();
```

| Paso 5: Ejecutar dotnet run y comparar los tiempos.

Resultado esperado: la consulta optimizada es más rápida y transfiere menos datos. La documentación XML explica las decisiones aplicadas.

| Analogía final

El checklist de rendimiento en una acería es como la lista de comprobaciones que el jefe de planta revisa antes de cada colada. Antes de encender el horno, comprueba la temperatura. Antes de laminar, comprueba el espesor. Antes de enviar, comprueba el peso. Cada comprobación evita un problema. El checklist de EF Core es igual: antes de ejecutar una consulta, se comprueba si se usa AsNoTracking, si se proyectan las columnas necesarias, si se evita el N+1, si se evita el producto cartesiano, si se aplican filtros y paginación, si se evitan funciones en Where, si se usan Compiled Queries y si se miden los tiempos. Cada comprobación evita un problema de rendimiento. La estrategia de optimización es como el plan de producción: se mide, se identifica, se aplica, se verifica y se documenta. La optimización es un ciclo continuo, no una tarea puntual. Así funciona el checklist de rendimiento en EF Core: se aplica a cada consulta y se documenta cada decisión.

| Resultado esperado

Al final del ejercicio, deberías haber:

Añadido los métodos del checklist a la interfaz IOrdenRepositorio.

Implementado los métodos en OrdenRepositorio con documentación XML.

Creado el caso de uso ChecklistRendimientoUseCase.

Registrado el caso de uso en el contenedor.

Insertado datos de prueba con veinte órdenes y planchas.

Ejecutado el checklist y las comparaciones de rendimiento.

Analizado la salida y comprobado el impacto de cada técnica.

Diagnosticado el error de eliminar AsSplitQuery.

Creado el método ObtenerOptimizadoCompleta con el checklist aplicado.

Resumen del estado del proyecto AceriaData al final del Módulo 4 Al final del Módulo 4, el proyecto AceriaData tiene:

La arquitectura limpia configurada en cuatro proyectos: AceriaData.Domain, AceriaData.Application, AceriaData.Infraestructure y AceriaData.Console.

El dominio con las entidades OrdenFabricacion, PlanchaAcero, Aleacion, EstadoOrden, DetalleOrden, CertificadoCalidad y OrdenAleacion.

La aplicación con las interfaces IOrdenRepositorio y IUnidadDeTrabajo, los casos de uso de todos los módulos y los DTOs.

La infraestructura con el AceriaDbContext, las configuraciones de Fluent API, las implementaciones de repositorios, la unidad de trabajo, las consultas compiladas y el observador de diagnóstico.

La consola con el método Main y la configuración del contenedor de dependencias.

El modelo de datos completo con relaciones uno a muchos, uno a uno y muchos a muchos.

Las claves primarias, alternativas y compuestas configuradas.

Los índices y las restricciones configurados.

El Soft Delete implementado con filtros globales.

La arquitectura limpia aplicada con la regla de dependencia respetada.

Las consultas LINQ con filtros, ordenaciones, proyecciones, agregaciones, agrupaciones, joins, carga Eager, carga Lazy, carga Explicit y composición de consultas.

Las buenas prácticas de acceso a datos aplicadas.

La optimización del rendimiento aplicada: análisis del SQL, Tracking vs No Tracking, resolución de identidad, solución al N+1, over-fetching, consultas ineficientes, Split Queries, Compiled Queries, paginación eficiente, diagnóstico y checklist de rendimiento
